

全局布局设计

1. 目标模型

采用文献[1]的全局布局算法。设计目标是在密度约束下得到最小线长，目标函数如下：

$$f(x,y)=\min(\sum_{e \in E} WL(e;x,y)) + \lambda D(x,y) \quad (1)$$

其中， e 表示单个网络， E 则表示整个电路的所有网络； x, y 为网络中单元的位置矢量。 $WL(e;x,y)$ 指单个网络的 HPWL， $D(\cdot)$ 是密度惩罚项，权重 λ 用于控制密度约束。

输入：def 文件中的电路放置单元信息，包括单元放置位置以及引脚连接等。

输出： x, y 矢量

在优化过程中， λ 的值会逐渐增加，从一开始主要关注线长最小化，逐渐过渡到同时考虑线长和密度的优化。

2. 最优化方法

目标函数 $f(x,y)$ 的求解采用 Nesterov (Nesterov's Accelerated Gradient, NAG) 加速梯度法。该方法属于梯度下降法 (Gradient Descent Algorithm) 一类的方法。梯度下降法是优化领域的基础算法，其核心思想是沿目标函数负梯度方向迭代更新参数，每次迭代步长由学习率控制，但在高维非凸场景中易陷入局部最优、收敛缓慢；

2.1 梯度下降法

一、基本原理

假设我们有一个目标函数 $J(\theta)$ ，其中 θ 是我们要优化的参数向量。我们的目标是找到一组参数 θ ，使得目标函数 $J(\theta)$ 达到最小值（如果是最大化问题，可以将其转化为最小化问题）。

梯度是一个向量，它指向函数在某一点上升最快的方向。对于函数 $J(\theta)$ ，其在点 θ 处的梯度记为 $\nabla J(\theta)$ 。在二维情况下 $\nabla J(\theta) = [\frac{\partial J(\theta)}{\partial \theta_1}, \frac{\partial J(\theta)}{\partial \theta_2}]$ ，其中 $\theta = [\theta_1, \theta_2]$ 。在高维情况下，梯度向量有更多的分量。

算法思想

梯度下降的基本思想是沿着目标函数的负梯度方向进行迭代搜索，因为负梯度方向是函数值下降最快的方向。每次迭代时，更新参数 θ ，使其朝着可能使目标函数值更小的方向移动。

二、算法流程

● 初始化

首先，随机初始化参数 θ 的值。例如，在一个简单的线性回归问题中，如果有两个参数 θ_0 和 θ_1 ，可以随机赋予它们一些初始值，如 $\theta_0 = 0$ ， $\theta_1 = 0$ 。

● 迭代更新

在每次迭代 t 中，按照以下公式更新参数：

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

其中， α 是学习率，它决定了每次迭代时参数移动的步长。如果 α 太小，收敛速度会很慢；如果太大，可能会导致算法无法收敛甚至发散。

● 停止条件

迭代过程会持续进行，直到满足某个停止条件。常见的停止条件包括：

- 达到最大迭代次数。例如，设定最多迭代 1000 次。
- 目标函数的变化小于某个阈值。
- 梯度的范数小于某个阈值。

三、算法描述

1. 初始化参数：

- 选择一个初始点 x_0 。
- 设置学习率 α (步长)，通常是一个小的正数。
- 设置最大迭代次数 T 或收敛阈值 ϵ 。

2. 迭代更新：

- 对于 $t = 0, 1, 2, \dots, T - 1$:
 1. 计算当前点的梯度： $g_t = \nabla f(x_t)$ 。
 2. 更新参数： $x_{t+1} = x_t - \alpha g_t$ 。
 3. 检查收敛条件：如果 $\|x_{t+1} - x_t\| < \epsilon$ 或 $\|g_t\| < \epsilon$ ，则停止迭代。

2.2. Nesterov 加速梯度法

Nesterov 加速梯度法作为其改进版，它在梯度下降法的基础上引入了动量（momentum）的概念，以加速收敛并减少振荡。动量项考虑了之前梯度的方向和大小，使得参数更新更加平滑，从而加速收敛并减少振荡，在理论上达成 $O(1/k^2)$ 收敛速率，既保持梯度下降的直观性与工程易实现性，又通过动量与前瞻机制加速收敛。

Nesterov 的算法描述：

NAG 的参数更新公式如下：

1. 计算当前梯度： $g_t = \nabla f(x_t)$
2. 计算动量： $v_t = \beta v_{t-1} + (1 - \beta)g_t$
3. 更新参数： $x_{t+1} = x_t - \alpha v_t$

其中：

- x_t 是第 t 次迭代的参数向量。
- g_t 是第 t 次迭代的目标函数梯度。
- v_t 是第 t 次迭代的动量向量。
- α 是学习率（步长）。
- β 是动量系数，通常取值在 0 到 1 之间。

3. 目标求解

采用 Nesterov 的框架进行迭代，逐渐逼近目标。

3.1 迭代的停止判定

- (1) 满足如下一个条件则停止迭代：
 - 密度溢出率 τ 小于目标密度溢出率 $\tau_{\min}$
 - 迭代次数大于规定最大迭代次数
- (2) 密度溢出率 τ 计算方法：

$$\tau = \frac{\text{总溢出面积}}{\text{归一化总单元面积}}$$

其中，

总溢出面积 = $\sum_{b \in B}$ 单网格的溢出面积_b，B 为所有划分网格。

归一化总单元面积 = 标准单元总面积 + 宏单元总面积 * *targetDensity*

其中，

单网格的溢出面积_b = b 网格布局元件面积 - *targetDensity* * 网格面积

其中 b 网格布局元件面积 = b 网格中(标准单元 + 宏单元 + 终端 + 暗节点)面积

3.2 单步迭代

为求得在密度约束下得到最小线长，即公式（1），在每次单步迭代中，首先计算目标函数原始梯度以及梯度矫正，得到矫正后梯度再计算步长。步长通过 Lipschitz 常数初始化，并通过回溯法找到最优解步长。确定步长之后更新布局，循环此过程直至目标函数收敛。

步骤 1：计算梯度 $\nabla f(x,y)$

对公式（1）求导，得到目标函数梯度：

$$\nabla f(x,y) = \nabla WL(x,y) + \lambda \nabla D(x,y). \quad (2)$$

要计算总梯度 ∇f ，则需要分别计算线长梯度 ∇WL 和密度梯度 ∇D ，再将结果线性相加。

步骤 1-1：计算线长梯度 $\nabla WL(x,y)$

公式（2）中线长梯度 $\nabla WL(x,y)$ 等于 WL 对 x、y 分别求导后再相加：

$$\nabla WL(x,y) = \sum_{e \in E} \left(\frac{\partial WL_e}{\partial x} + \frac{\partial WL_e}{\partial y} \right). \quad (3)$$

布局被划分为多个网络，E 为被划分网络集合； $\frac{\partial WL_e}{\partial x}$ 是单个网络线长 WL_e 在 x 方向的导数； $\frac{\partial WL_e}{\partial y}$ 是单个网络线长 WL_e 在 y 方向的导数。

一个网络中存在多个引脚，单个网络线长梯度等于网络中所有引脚梯度之和：

$$\frac{\partial WL_e}{\partial x} = \sum_{i \in e} \frac{\partial WL_e}{\partial x_i}, \quad \frac{\partial WL_e}{\partial y} = \sum_{i \in e} \frac{\partial WL_e}{\partial y_i}. \quad (4)$$

对于网络中的单个引脚 i，线长梯度计算公式为：

$$\begin{aligned} \frac{\partial WL_e}{\partial x_i} &= \frac{\left(1 + \frac{x_i}{\gamma}\right) b_e^+ - \frac{1}{\gamma} c_e^+}{(b_e^+)^2} \cdot a_i^+ - \frac{\left(1 - \frac{x_i}{\gamma}\right) b_e^- + \frac{1}{\gamma} c_e^-}{(b_e^-)^2} \cdot a_i^-, \\ \frac{\partial WL_e}{\partial y_i} &= \frac{\left(1 + \frac{y_i}{\gamma}\right) b_e^+ - \frac{1}{\gamma} c_e^+}{(b_e^+)^2} \cdot a_i^+ - \frac{\left(1 - \frac{y_i}{\gamma}\right) b_e^- + \frac{1}{\gamma} c_e^-}{(b_e^-)^2} \cdot a_i^-. \end{aligned} \quad (5)$$

其中， γ 是控制 WL 逼近实际线长的平滑度和精度的参数，通过以下方法计算：

$$\gamma = 8.0w_b \times 10^{20/9 \times (\tau - 0.1) - 1.0}. \quad (6)$$

公式（5）中， w_b 为单个网格 b 的宽度； τ 为密度溢出率。

b_e^+ 、 c_e^+ 、 b_e^- 、 c_e^- 、 a_i^+ 、 a_i^- 的计算公式如下表格：

Notation	Description	Notation	Description
x_e^+	$\max_{i \in e} x_i, e \in E$	x_e^-	$\min_{i \in e} x_i, e \in E$
a_i^+	$e^{\frac{x_i - x_e^+}{\gamma}}, \forall i \in e, e \in E$	a_i^-	$e^{\frac{x_i - x_e^-}{\gamma}}, \forall i \in e, e \in E$
b_e^+	$\sum_{i \in e} a_i^+, \forall e \in E$	b_e^-	$\sum_{i \in e} a_i^-, \forall e \in E$
c_e^+	$\sum_{i \in e} x_i a_i^+, \forall e \in E$	c_e^-	$\sum_{i \in e} x_i a_i^-, \forall e \in E$

步骤 1-2： 计算密度梯度 $\nabla D(x,y)$

将每个单元（标准单元、宏单元）建模为带电荷的粒子，电荷大小 q 等于单元的面积；密度 D 被建模为总电势能，电势 ψ 的梯度取负数就是电场 ξ 。电荷间相互排斥，在电场力作用下将单元分开，通过最小化系统总电势能，最终达到静电平衡状态，即密度均匀分布。

对于点电荷系统，在已知电势的情况下，电势能=电荷*电势，即 $D=q*\psi$ 。

公式（2）中密度梯度 $\nabla D(x,y)$ 等于密度对 x,y 分别求导再相加：

$$\nabla D(x,y) = \frac{\partial D}{\partial x} + \frac{\partial D}{\partial y}. \quad (7)$$

其中， $\frac{\partial D}{\partial x}$ 为密度在 x 方向上的梯度， $\frac{\partial D}{\partial y}$ 为密度在 y 方向上的梯度。

布局中存在多个单元，密度梯度等于每个单元密度梯度之和，即：

$$\frac{\partial D}{\partial x} = \sum_{i \in V} \frac{\partial D}{\partial x_i}, \quad \frac{\partial D}{\partial y} = \sum_{i \in V} \frac{\partial D}{\partial y_i}. \quad (8)$$

V 为布局中所有单元集合。

单元 i 密度梯度计算公式为：

$$\frac{\partial D}{\partial x_i} = -q_i \xi_{i_x}, \quad \frac{\partial D}{\partial y_i} = -q_i \xi_{i_y}. \quad (9)$$

其中 q_i 为单元 i 的电荷大小，它等于单元 i 的面积。布局区域被划分为 $m \times m$ 个网格，通过对每个网格内的标准单元、宏单元、填充单元的单元面积求和，得到单元 i 面积 q_i 。 ξ_x 为电场 ξ 在 x 方向值， ξ_y 为电场 ξ 在 y 方向值。

电势梯度 ξ_x 、 ξ_y 计算公式为：

$$\xi_x(x, y) = \sum_{j=0}^{m-1} \sum_{k=0}^{m-1} \frac{a_{j,k} w_j}{w_j^2 + w_k^2} \sin(w_j x) \cos(w_k y)$$

$$\xi_y(x, y) = \sum_{j=0}^{m-1} \sum_{k=0}^{m-1} \frac{a_{j,k} w_k}{w_j^2 + w_k^2} \cos(w_j x) \sin(w_k y) \quad (10)$$

其中， w_j 、 w_k 是频率分量， $a_{j,k}$ 是密度 ρ 的 DCT 系数：

$$a_{j,k} = \frac{1}{m^2} \sum_{x=0}^{m-1} \sum_{y=0}^{m-1} \rho(x, y) \cos(w_j x) \cos(w_k y). \quad (11)$$

$m*m$ 表示布局区域被划分网格数， ρ 表示网格中的单元密度，即网格内单元总面积\网格面积。

公式（10）的推导思路：用密度 $\rho(x, y)$ 分布与电势 ψ 的关系，通过密度 $\rho(x, y)$ 求电势 ψ ，再对 ψ 求导得到电场力 ξ ，其中，密度 $\rho(x, y)$ 为已知量。

步骤 1-2-1：统计各种单元的网格密度贡献

对每一个节点（包括标准单元、宏单元、填充单元），统计与之有重叠的网格 $\text{bin}[x][y]$ ，并对每一个网格计算节点密度：

- 标准单元：

$\text{bins}[i][j] \rightarrow \text{nodeDensity} += \text{LengthScale.x} * \text{LengthScale.y} * \text{overlapArea};$

- 宏单元：

$\text{bins}[i][j] \rightarrow \text{nodeDensity} += \text{LengthScale.x} * \text{LengthScale.y} * \text{targetDensity} * \text{overlapArea};$

- 填充单元：

$\text{bins}[i][j] \rightarrow \text{fillerDensity} += \text{LengthScale.x} * \text{LengthScale.y} * \text{targetDensity} * \text{overlapArea};$

其中， overlapArea 为该节点与网格的重叠面积， LengthScale.x 和 LengthScale.y 分别为 x 和 y 方向的长度拉伸，默认为 1，即不拉伸，当节点尺寸(x 或 y 方向) 小于网格大小时，

$$\text{LengthScale} = \text{节点尺寸} / \text{网格大小}$$

步骤 1-2-2：计算网格的总密度

对每一个网格： $\text{eDensity} = \text{总单元面积} / \text{网格面积}$

其中，总单元面积 = $\text{bins}[i][j] \rightarrow \text{nodeDensity} + \text{bins}[i][j] \rightarrow \text{darkDensity} + \text{bins}[i][j] \rightarrow \text{fillerDensity} + \text{bins}[i][j] \rightarrow \text{terminalDensity}$ ， darkDensity 与 terminalDensity 在初始化时求得，详见初始布局设计 3.2 节。

步骤 1-2-3: 计算 DCT 系数 $a_{j,k}$

对于空间密度分布 $\rho(x, y)$, 将其通过 DCT 转换到频域, 其 DCT 系数 $a_{j,k}$ 为:

$$a_{j,k} = \frac{1}{m^2} \sum_{x=0}^{m-1} \sum_{y=0}^{m-1} \rho(x, y) \cos(w_j x) \cos(w_k y) \quad (20)$$

其中, w_j, w_k 为频率分量, 具体数值为 $w_j = \frac{j\pi}{m}$, $w_k = \frac{k\pi}{m}$; m 为划分网格数。

步骤 1-2-4: 计算逆 DCT 系数 $\rho(x, y)$

逆 DCT 过程将频域信号转换为原始空间密度分布:

$$\rho(x, y) = \sum_{j=0}^{m-1} \sum_{k=0}^{m-1} a_{j,k} \cos(w_j x) \cos(w_k y) \quad (21)$$

其中, w_j, w_k 为频率分量, 其数值为 $w_j = \frac{j\pi}{m}$, $w_k = \frac{k\pi}{m}$; m 为划分网格数。

步骤 1-2-5: 计算电场 ξ_x, ξ_y

对 $\rho(x, y)$ 求导再取反, 得到电场 ξ_x, ξ_y 。

$$\begin{aligned} \xi_x(x, y) &= \sum_{j=0}^{m-1} \sum_{k=0}^{m-1} \frac{a_{j,k} w_j}{w_j^2 + w_k^2} \sin(w_j x) \cos(w_k y) \\ \xi_y(x, y) &= \sum_{j=0}^{m-1} \sum_{k=0}^{m-1} \frac{a_{j,k} w_k}{w_j^2 + w_k^2} \cos(w_j x) \sin(w_k y). \end{aligned}$$

其中, $a_{j,k}$ 是密度 ρ 的 DCT 系数。

注: 其中第 (4) 和第 (5) 步可以用 FFT 包中的逆 SCT, 逆 CST 来完成。

步骤 2: 梯度矫正

在同时有宏单元与标准单元的混合布局中, 因为宏单元与标准单元的物理尺寸、面积差异较大, 导致求解梯度时不平衡: 宏单元面积较大, 梯度幅值高, 容易在布局边界处来回移动。密度预处理的目标是通过数学变换, 将不同尺寸的物体在优化空间中 “标准化”, 使其梯度影响趋于一致, 从而加速收敛并提升布局质量。

将原始梯度 ∇f 变换后得到预条件化梯度:

$$\nabla f_{\text{pre}} = \tilde{H}_f^{-1} \nabla f. \quad (25)$$

按文献推导， $\tilde{H}_f$ 为目标函数的二阶导数，公式（25）近似变为：

$$\nabla f_{\text{pre}} = \nabla f / (|E_i| + \lambda q_i)$$

其中线长二阶导数近似为 $|E_i|$ ，它为节点 i 所在的网络数目，密度函数的二阶导数近似为 q_i ，它为节点 i 的单元面积。

步骤 3：梯度下降

步骤 3-1 计算步长 $\hat{\alpha}_k$

计算 Lipschitz:

$$\tilde{L}_k = \frac{\|\nabla f_{\text{pre}}(\mathbf{v}_k) - \nabla f_{\text{pre}}(\mathbf{v}_{k-1})\|}{\|\mathbf{v}_k - \mathbf{v}_{k-1}\|} \quad (30)$$

$$\hat{\alpha}_k = 1/\tilde{L}_k \quad (31)$$

步骤 3-2 计算动量系数 β_{k+1}

$$\beta_{k+1} = \left(1 + \sqrt{4\beta_k^2 + 1}\right)/2 \quad (32)$$

其中 β_{k+1} 的初值为 1；

步骤 3-3 迭代预测位置

3-3-1 对每一个节点，作：

3-3-1-1：计算单元位置 $\hat{\mathbf{u}}_{k+1}$ ：

$$\hat{\mathbf{u}}_{k+1} = \mathbf{v}_k - \hat{\alpha}_k \cdot \nabla f_{\text{pre}}(\mathbf{v}_k) \quad (33)$$

其中 $\mathbf{v}_k$ 为中间变量，作参考位置。

3-3-1-2：计算参考单元位置 $\hat{\mathbf{v}}_{k+1}$ ：

$$\hat{\mathbf{v}}_{k+1} = \hat{\mathbf{u}}_{k+1} + \frac{\beta_k - 1}{\beta_{k+1}} (\hat{\mathbf{u}}_{k+1} - \mathbf{u}_k) \quad (34)$$

3-3-2：用 $\hat{\mathbf{v}}_{k+1}$ 更新当前单元的位置；

3-3-3：按步骤 1 和 2，计算梯度；

3-3-4：计算步长 $\hat{\alpha}_{k+1}$

$$\hat{\alpha}_{k+1} = \frac{\|\hat{\mathbf{v}}_{k+1} - \mathbf{v}_k\|}{\|\nabla f_{\text{pre}}(\hat{\mathbf{v}}_{k+1}) - \nabla f_{\text{pre}}(\mathbf{v}_k)\|}$$

3-3-5：判断循环条件

$$\hat{\alpha}_k > \epsilon \hat{\alpha}_{k+1} \quad (35)$$

其中 ϵ 设定为小于 1 的数。若 (35) 式成立，则 $\hat{\alpha}_k = \hat{\alpha}_{k+1}$ ，并跳转至步骤 3-3，否则执行步骤 3-4。

步骤 3-4 更新动量系数

$$\beta_k = \beta_{k+1} \quad (36)$$

步骤 4：更新 lammda

$$\lambda_k = \mu_k \lambda_{k-1} \quad (37)$$

其中 $\mu_k = 1.1^{-(\Delta HPWL_k / \Delta HPWL_{REF}) + 1.0}$ ； $\Delta HPWL_{RFF} = 3.5 \times 10^5$ ， $\Delta HPWL_k = HPWL(\vec{v}_k) - HPWL(\vec{v}_{k-1})$ 。

通过公式 (37) 动态更新 λ ，可以平衡线长和密度的优化优先级。

3.3 输出线长

HPWL (Half-Perimeter Wirelength) 是布局中常用的线长估计方法，用于快速评估电路模块间的连线复杂度。其核心思想是通过计算模块边界框 (Bounding Box) 的周长的一半来近似线长。

HPWL 计算公式为：

$$HPWL = \sum_{e \in E} (\max_{i,j \in e} |x_i - x_j| + \max_{i,j \in e} |y_i - y_j|) . \quad (38)$$

其中， e 表示单个网络， x_i 表示该网络中的引脚 i 的 x 方向坐标， y_i 表示该网络中的引脚 i 的 y 方向坐标。

实现：

对每一个 net

{

 遍历所有 pin,

 { 寻找 minX、maxX、minY、maxY; }

}

HPWL = ((maxX - minX) + (maxY - minY))

参考文献

- [1] J. Lu, P. Chen, C.-C. Chang, L. Sha, D. J.-H. Huang, C.-C. Teng, and C.-K. Cheng, “ePlace: Electrostatics-based placement using fast fourier transform and nesterov’s method,” ACM TODAES, vol. 20, no. 2, p. 17, 2015.
- [2] J. Lu, H. Zhuang, P. Chen, H. Chang, C.-C. Chang, Y.-C. Wong, L. Sha, D. Huang, Y. Luo, C.-C. Teng et al., “ePlace-MS: Electrostatics-based placement for mixed-size circuits,” IEEE TCAD, vol. 34, no. 5, pp. 685–698, 2015.